

SceneIt

Multimodal Movie Recommendations with Deep Learning

Tim Pang
Georgia Tech
tpang33@gatech.edu

Michael Bock
Georgia Tech
michaelb18@gatech.edu

Harrison Zhao
Georgia Tech
hzhao374@gatech.edu

Abstract

Deep learning recommendation models (DLRMs) have emerged as a powerful framework for combining user and item features in large-scale personalization tasks. In this work, we extend the DLRM architecture to a multimodal setting for movie rating prediction, incorporating structured metadata, plot descriptions, and poster images. Using pre-trained BERT and ResNet backbones, we frame the task as a regression problem and train on the MovieLens dataset. Our model achieves a de-normalized RMSE of 0.7042, outperforming baselines such as SVD and user/item-based collaborative filtering. Prediction errors are approximately Gaussian with a mean of 0.0235 and median of -0.0023. Ablation studies reveal that metadata alone yields the lowest unimodal RMSE (0.871), while combining metadata with either text (0.855) or image (0.845). Surprisingly, using all three modalities results in a higher RMSE (0.864), suggesting that naive fusion may underperform without more expressive modeling. These findings support the value of multimodal features in recommender systems and motivate future work in fusion strategies.

1. Introduction

Recent internet trends have revealed an immense growth in social networks and streaming content, prompting the development of recommender systems as a way to curate content for users. Historically, competitions such as the Netflix Prize and Movie Lens competitions began a focus on creating recommender systems for movies, from which methods such as content-based filtering emerged, which learn unsupervised embeddings during training and cluster similar items and users together [9]. Another popular framework is collaborative filtering, which decomposes item-user interaction matrices to fill a sparse matrix, which can then be used to recommend new items. Several methods such as singular value decomposition (SVD), and Neural Collaborative Filtering (NCF) [5] are used to factor matrices for

collaborative filtering. Other methods use a combination of collaborative and content filtering to achieve state of the art results. Deep Learning Recommender Model (DLRM) [7] uses matrix decomposition inside of a neural network to predict the click-through rate of items.

Large companies (e.g., Netflix, Amazon) commonly store extensive metadata about movies including length, aspect ratio, director, genre, and much more that models such as NCF and DLRM have used to improve results. Networks such as DLRM and Wide and Deep [3] include categorical features such as genre and dense numerical features such as length. The Two Towers model [9] explicitly includes multiple feature processors to combine features from different data sources. However, with the explosion in information collected by websites such as IMDb, more information about movies became available. Now, movies have extensive data, such as posters, short trailers, metadata, and plot summaries. Most architectures discussed, such as DLRM and Wide and Deep, combine different types of metadata but do not take advantage of the additional information provided by databases such as IMDb. Using a multimodal recommendation system that takes in more information could yield more accurate predictions of what a user will rate a movie.

In this paper, we propose a multimodal movie recommendation system inspired by DLRM to improve how movie recommendation systems predict what users will enjoy. Our goal was to build a system that can better understand both users and movies by using more information than traditional methods. By combining user embeddings with movie representations derived from metadata, plot summaries, and poster images, our model was able to achieve a de-normalized RMSE of 0.7042 on the MovieLens 100k dataset, with a mean prediction error of 0.0235 and a median error of -0.0023. Compared to baseline models, our multimodal model (RMSE = 1.2070) outperformed Singular Value Decomposition (SVD) (RMSE = 1.301), user Collaborative Filtering (CF) (RMSE = 1.4347), and item-CF (RMSE = 1.3555). An ablation study showed that metadata alone provided the strongest single-modality signal (RMSE

= 0.871), while combining metadata with plots (RMSE = 0.845) or with posters (RMSE = 0.855) improved performance. However, combining all three modalities resulted in slightly worse performance (RMSE = 0.864), suggesting the need for more sophisticated fusion strategies.

Improving movie recommendation through multimodal information can lead to more accurate, personalized, and engaging user experiences. It offers the potential to recommend not just popular or heavily rated content, but also new, unfamiliar movies based on their intrinsic characteristics rather than solely user behavior.

2. Data

2.1. Datasets

Our multimodal model uses a combination of metadata, plot descriptions, images of posters, and user ratings drawn from several publicly available movie-related datasets. Metadata information for movies was extracted from the IMDb5000 dataset [1], which includes 28 features and contains both categorical and numerical metadata, such as genre and IMDb rating, for 5000 unique movies.

Plot descriptions were contained in the Wikipedia Movie Plots dataset [6]. This dataset contains descriptions for roughly 35000 movies, offering natural-language English descriptions of each movie’s plot. Images of movie posters were found in the Movie Genre from its Poster dataset [8]. The dataset has images of covers for over 39000 movies.

Lastly, user data was pulled from the MovieLens Small dataset [4]. This dataset is comprised of 100k user ratings for movies in the database and is a standard used for model evaluation. Each rating is organized by a user’s userID, the movieID of the movie reviewed, and a numerical rating between 0.5 and 5, in increments of 0.5.

2.2. Pre-processing

We used four different data sets to train our model, and carefully applied a sequence of pre-processing steps to ensure that our data would be comprehensive and meaningful. Each dataset was joined using IMDb IDs or movie titles when the IMDb IDs were unavailable. Several columns that were redundant or did not contribute meaningfully to the data were dropped (e.g., movie IMDb link and colors). One-hot encoding was applied to the genres column to more easily interpret the different combinations of genre per movie.

Missing fields were imputed either with zeros or the median value across the dataset, depending on the column’s nature. Categorical columns were assigned ids from 1 to N , where N is the number of possible values for a categorical column, and numerical columns were standardized using a standard scaler. User ratings, originally on a [0.5,5.0] scale, were normalized to [0,1] during training and later denormalized for analysis.

Images (movie posters) were downloaded using URLs from our poster dataset and resized to 224×224 . Any missing posters were saved as RGB(0,0,0) in order to provide a neutral baseline for image features.

We merged the metadata, poster images, and plot descriptions using a combination of IMDb ID, normalized movie title, and release year to maximize matching accuracy. The MovieLens rating dataset was used to pair each user rating to a movie using the movieID and imdbID. Our final merged dataset contained approximately 3,832 movies with complete multimodal information.

3. Approach

Our approach for this project was to create a multimodal model specifically aimed towards predicting user ratings for movies they have not yet seen. We combined text, images, and metadata embeddings and used them as inputs into our Deep Learning Recommendation Model (DLRM), originally introduced by Naumov et al [7]. This approach has shown to be effective in its predictive power, achieving accuracy scores of nearly 0.8 on the Criteo Ad Kaggle data set, outperforming a Deep and Cross network without extensive tuning [7]. Many different media streaming services use this recommendation approach, including Meta (where DLRM was invented), Netflix, and Amazon.

While our model architecture is inspired by the original paper’s model architecture, it significantly extends the traditional DLRM design by incorporating additional unstructured modalities. We use BERT embeddings for movie plot descriptions and ResNet embeddings for movie poster images, in addition to structured metadata features. Instead of applying a dot-product interaction layer, we concatenate all modality embeddings and pass them through an MLP for prediction. Furthermore, whereas DLRM is designed for binary classification (e.g., click prediction), we frame our task as a regression problem to predict continuous user rating scores.

3.1. Model Architecture

3.1.1 Problem Structure

Our problem was structured as a regression task, predicting a continuous user rating for a given movie based on multimodal information. Each instance involved a user ID and associated movie features (structured metadata, plot description, and poster image).

The structure of our model directly reflected this design (Figure 1). Each modality was embedded separately: user IDs were passed through a learnable embedding table; plot descriptions were encoded using a pretrained BERT model; poster images were processed through a pretrained ResNet-50 feature extractor; and structured metadata was projected through a fully connected layer. The outputs of

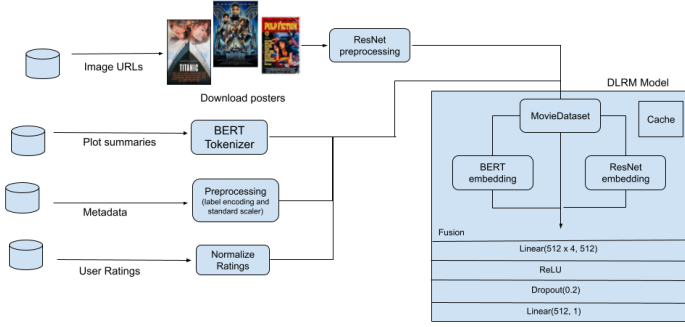


Figure 1. Our multimodal DLRM-inspired architecture combining user embeddings, BERT plot embeddings, ResNet poster embeddings, and structured metadata.

these components were concatenated to form a fused embedding vector, which was passed through a multilayer perceptron (MLP) consisting of a hidden layer with ReLU activation, dropout regularization, and a final output layer predicting a single scalar rating. This modular architecture closely mirrored the multimodal structure of the input data.

3.1.2 Learned Parameters

Learned parameters in the model included the user embeddings, all projection layers (for plot, poster, and metadata embeddings), and the MLP fusion layers. The BERT encoder and ResNet-50 feature extractor were kept frozen during training to prevent overfitting and to leverage powerful pretrained representations without requiring large-scale finetuning.

3.1.3 Pre and Post-processing

Each modality was preprocessed to match the input format expected by its respective encoder: user IDs were integer-encoded for embedding lookup; plot descriptions were tokenized into input IDs and attention masks for BERT; poster images were resized to 224×224 and normalized per ResNet standards; and metadata features were label-encoded and standardized.

The target output was a normalized user rating in the $[0, 1]$ range, corresponding to an original rating scale of $[0.5, 5.0]$ stars. Post-processing involved de-normalizing predicted ratings back into star values for evaluation and analysis.

3.1.4 Existing models

We built our model using the PyTorch deep learning framework. Pretrained components such as BERT and ResNet were adopted to accelerate development and improve generalization. Our model was inspired by the original DLRM

model outlined by Naumov et al, albeit with significant changes as discussed previously [7].

3.2. Training Setup

We used a train-test split of 0.8/0.2 and used a custom `MovieDataset` Pytorch Dataset class with a `PyTorch DataLoader` to handle each training batch efficiently. Each batch contained sampled user IDs, metadata, poster images, and plot descriptions and BERT tokenized inputs for training.

We used a stratified split for the training/test sets, as this would allow users to be present in both sets, which would allow us to more easily predict a user’s preferences.¹

3.2.1 Hyperparameters

Due to the large volume of embedding data, we tuned our model to balance training cost and performance, selecting 20 epochs to enable efficient yet effective training. For our metadata dimension, we used 28, corresponding to the number of metadata columns in our dataset (e.g., directors, actors, genres, etc.). Our embed dimension was chosen as 512 to balance model expressiveness and computational efficiency. Our number of unique users was 610, which was obtained from our ratings dataset. A batch size of 32 allowed the model to capture sufficient diversity in user-movie interactions while fitting within GPU memory constraints. A learning rate of 1×10^{-4} was used to allow the model to learn quickly given the relatively low number of epochs, while maintaining stable optimization. Our optimizer was Adam, due to its adaptive learning rate capabilities, which generally perform well across diverse deep learning tasks.

3.2.2 Loss Function

For our model, we used Mean Squared Error (MSE) as our loss function, as we were framing it as a regression problem and trying to obtain the loss between our predicted scores and the actual user scores. MSE penalizes larger deviations between predicted and true ratings more heavily, making it an appropriate choice for minimizing rating prediction errors across users and movies.

3.2.3 Deep Learning Framework

We implemented our model using the PyTorch deep learning framework. PyTorch provided flexible dynamic computation graphs, seamless integration of pretrained models (BERT and ResNet), and efficient handling of multimodal inputs through custom datasets and data loaders.

¹A non-stratified split would be a good option if we wanted to explore the cold start problem, in which a user begins with no pre-existing preferences. [2]

4. Experiments and Results

We trained our multimodal recommendation model and compared the predicted ratings against the actual user ratings, obtaining meaningful statistical evidence of the model’s predictive capabilities. We further conducted comparative experiments against baseline models, such as Singular Value Decomposition (SVD) and Collaborative Filtering (CF), as well as ablation studies to assess the contribution of each individual modality.

4.1. Evaluation Metrics and Measurement of Success

We measured success primarily through two metrics: the Mean Squared Error (MSE) loss during training and validation, and the Root Mean Squared Error (RMSE) between predicted and true ratings on the validation set. MSE directly reflects how well the model minimizes the average squared difference between predicted and actual ratings, while RMSE provides an interpretable measure of average prediction error in the normalized $[0, 1]$ rating space.

Our goal was to achieve a low validation RMSE, stable convergence behavior across epochs, and good generalization without severe overfitting.

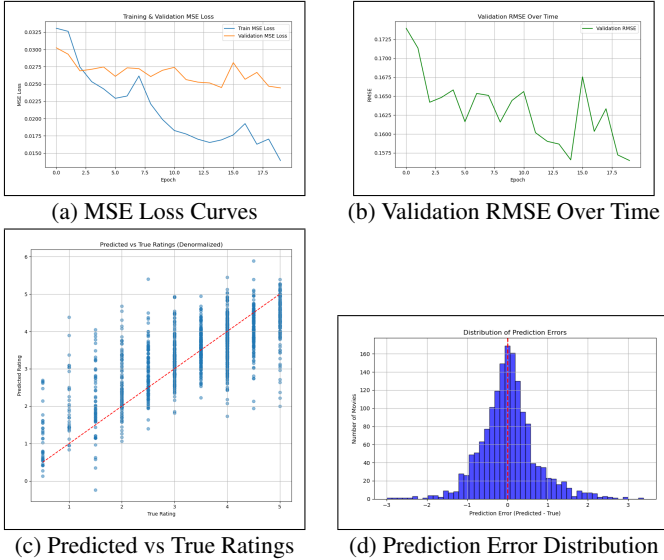


Figure 2. Summary of training behavior. MSE Loss and Validation RMSE were normalized, and the prediction charts were denormalized.

4.2. Loss Curves and Convergence

Figure 2(a) shows that training MSE steadily decreases, while validation MSE oscillates slightly but ultimately stabilizes, indicating successful learning without significant overfitting. Some spikes were observed in both the training and validation loss curves, particularly in later epochs; however, these fluctuations remained within a small range.

The validation curve did not decrease as significantly as the training curve, but it did show a .06 loss overall, meaning that it was able to generalize, but additional hyperparameter tuning, regularization techniques, or architectural adjustments could further improve generalization performance.

4.3. Validation RMSE

Figure 2(b) shows the Validation RMSE over time. It tends to oscillate but has a general downward trend, peaking at epoch 20 with an RMSE of 0.1571, or 1.2065 after de-normalizing the score. The oscillation could be due to a number of factors, including noise in the multimodal inputs, insufficient regularization, or sparseness in the user/move interactions. Since the loss jumped up at certain points (see epoch 15), this could potentially indicate the model was beginning to overfit; this can also be seen in the Figure 2(a) chart which also shows a similar increase at that time slice. Although the oscillations appear prominent visually, they remain within a narrow range—typically not exceeding 0.01. To mitigate this variability, future iterations could experiment with larger validation sets for better averaging, increased dropout to improve robustness, or the use of smoothed metrics such as a moving average of recent RMSE values.

4.4. Prediction Error Analysis

Our model reported a de-normalized RMSE of 0.7042 stars, indicating that on average, the model’s prediction deviated from the true user ratings by about 0.7 stars. Figures 2(c, d) show the relationship between the predicted and true ratings, as well as the distribution of prediction errors, respectively. Ratings in the MovieLens dataset range from 0.5 to 5.0 stars in increments of 0.5, resulting in a relatively coarse rating scale. The scatter plot demonstrates that the prediction errors are fairly consistent across the rating spectrum, without large systematic biases toward over-prediction or under-prediction. The histogram of prediction errors exhibits an approximately Gaussian (normal) distribution centered around zero, suggesting that the model’s errors are balanced and largely unbiased. The mean error was 0.0235 stars, and the median error was -0.0023 stars, indicating that on average, the model’s predictions were very close to the actual user scores, and that despite our earlier concerns, the model was able to generalize well.

4.5. Comparison with Baseline Models

The performance of our multimodal model is evaluated against 3 state-of-the-art recommendation systems: singular-value decomposition, item-based collaborative filtering (item-CF), and user-based collaborative filtering (user-CF). These 3 recommenders all use unary data, only using user ratings to make predictions, allowing us to investigate the effects of incorporating multimodal data in a

recommendation model.

4.5.1 Singular-value Decomposition

SVD is an algorithm that decomposes a matrix A into the product of 3 matrices, represented mathematically by the equation $A = U\Sigma V^T$. To be used as a movie recommender, matrix A is created by converting the user ratings data into a user-item matrix, where each row represents a user and each column represents the rating for a movie. Empty ratings are filled with a value of 0.5, the midpoint of the normalized rating space. This matrix is then decomposed using SVD into a user factors matrix (U), singular value matrix (Σ), and item factor matrix (V^T) that is used to predict missing ratings.

4.5.2 Item and User-based Collaborative Filtering

Item and user-CF are methods that use similarity measures to generate predicted ratings using user rating data. Similar to SVD, item and user-CF begin by converting user ratings into the user-item matrix A . Pairwise similarity measures are then calculated using this user-item matrix based on the type of collaborative filtering being performed. In item-CF, pairwise similarities are calculated between different movies, while similarities are calculated between different users in user-CF. These similarity matrices are then used to find movies or users that are similar to their own preferences to make predictions.

4.5.3 Results

A train-test split of 0.8/0.2 was once again used for this experiment, with all recommenders using the same train-test sets. A vanilla SVD model was fitted using the train set, while item and user-CF were initialized with cosine similarity as its similarity measure.

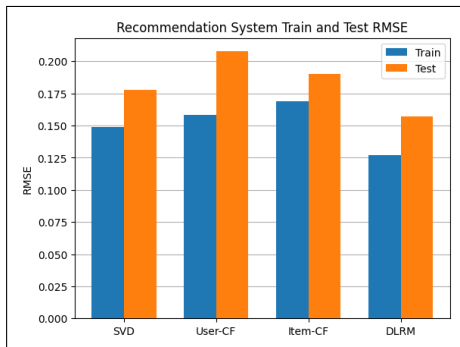


Figure 3. Normalized RMSE comparison of different recommenders. DLRM refers to the multimodal model presented in this paper.

Figure 3 shows the RMSE of each of the 4 recommendation systems. Notably, the multimodal model achieves the

lowest error, with final denormalized train and test RMSE values of 1.0724 and 1.2070 compared to 1.1687 and 1.301 for SVD, 1.2115 and 1.4347 for user-CF, and 1.2610 and 1.3555 for item-CF. Recall that SVD and collaborative filtering only take in unary data in the form of user ratings. The use of unary data results in a lack of rich feature representations that are present in our multimodal model, giving a likely explanation for the lower performance of the classical recommenders tested. Though not conclusive, we get strong evidence supporting our hypothesis that multimodal data can improve the generalization capability of our model.²

4.6. Ablation Study

We conducted a study where we remove each component of the model to see how the different backbones affected the model’s performance. For this study, we were able to link more plots to posters, so we were able to train on 9000 movies instead of the normal 5000. Using 9000 movies, we were able to achieve more reliable results because we had more movies that had all three of metadata, posters, and plots, so we would not be comparing models when some modalities were missing, which would have made certain modalities invalid. We train each configuration with the same hyperparameters as in [subsubsection 3.2.1](#), except we only train for 5 epochs.

Table 1 shows the results of this ablation study on the model. In general, as more modalities are added to the model, the RMSE gets lower, indicating improved rating prediction performance. Adding more modalities to the model provides a richer feature space for the model to use to predict ratings. Having a richer feature space means that each movie’s profile will be more unique, meaning that as the model builds its latent space, more detailed representations of movies will build each user’s profile. This will make each user’s profile more detailed, which will in turn help the model predict what the user would rate a new movie more accurately.

One exception to this is that the metadata fields provide the most information to the model. We can see from the three experiments that only user one modality, that using only the movie posters the RMSE is 1.05, using only the plots has an RMSE of 0.882, and using only the metadata has an RMSE of 0.871. This implies that the metadata provides the clearest features for the model. When we add multiple modalities, using only posters and plots, the RMSE is 0.944. When we include metadata instead of posters, the RMSE is 0.855 and when we include metadata instead of plots, the RMSE is 0.845.

An interesting result of this study is that the RMSE rises

²The normalized train and test values were, respectively, multimodal model: 0.1272 and 0.1571, SVD: 0.1486 and 0.1780, user-CF: 0.1581 and 0.2077, item-CF: 0.1691 and 0.1901.

Mode	Best Validation RMSE
Image, Text, and Metadata	0.864
Only Image and Text	0.944
Only Image and Metadata	0.845
Only Image	1.05
Only Text and Metadata	0.855
Only Text	0.882
Only Metadata	0.871

Table 1. RMSE Comparison Across Different Modalities

to 0.864 when all three modalities are used. To understand why, recall that the model uses a single linear layer and concatenation for the data fusion. When we include all three modalities, many more data points are introduced to the fusion layer. While training, it is required that a neural network is over-parameterized; it must have many more parameters than are required to solve the problem. One layer for fusion may not be enough. Either the fusion layer could not complete the fusion of all three modalities because there were not enough parameters. Models such as Wide and Deep first conduct fusion using a layer with many hidden units before using many more smaller linear layers. Increasing the number of layers after the fusion layer may improve the results from using all modalities by improving the quality of the fused latent representation of three modalities. Another solution for this problem would be to train the model for longer. It may take longer for a larger model to train. More data input to the fusion layer would require it to have more parameters, and a larger model may take more epochs to train than the smaller, two modality models. Observing the loss curves as well as the loss curve in [Figure 2](#), after 5 epochs, the validation loss still decreases for the three modality model.

5. Conclusion

In this project, we designed and implemented a multimodal recommendation system that combines structured metadata, poster images, and plot descriptions to predict user movie ratings. Our model builds upon a DLRM-inspired architecture, fusing user embeddings with multimodal movie representations, and was evaluated on the MovieLens dataset.

Our final model achieved a de-normalized RMSE of approximately 0.7042 stars, demonstrating strong predictive performance and generalization ability. Analysis of the loss curves and RMSE trends indicated that while some oscillations were present during training, these remained within acceptable bounds and did not suggest significant overfitting. Furthermore, the distribution of prediction errors was approximately Gaussian and centered around zero, confirming that the model’s predictions were balanced and unbiased.

We observed through comparative experiments against baselines such as Singular Value Decomposition (SVD) and Collaborative Filtering (CF), our multimodal model exhibited superior performance, showcasing the value of integrating diverse sources of movie information beyond historical user ratings alone.

The ablation study provided deeper insight into the contribution of each modality. Metadata emerged as the most informative single feature type, yielding the lowest RMSE when used alone. Combining metadata with either poster images or plot descriptions further reduced RMSE, indicating that multimodal representations enrich the model’s understanding of user preferences. However, a surprising result was that simply fusing all three modalities slightly degraded performance compared to two-modality combinations. This suggests that our simple concatenation-based fusion strategy, followed by a single linear layer, was insufficient to fully capture the joint structure across all modalities.

We ran into several unexpected instances: parts of our data were missing during the dataset merging process, particularly for movie posters, leading us to substitute blank placeholders for missing images. We did our best to fill in missing values with zeroes and median values, as described in the approach section. After cleaning and merging datasets, we ended up with 3832 movies total. When we combined with user ratings, we encountered a many-to-many relationship between users and movies, which necessitated careful handling to avoid redundant BERT and ResNet computations. To address this, we implemented caching mechanisms that significantly reduced training time.

5.1. Future Work

Based on our findings, future directions could include designing more sophisticated fusion architectures, such as multi-layer perceptrons after fusion, attention-based modality fusion, or gating mechanisms to dynamically weight modalities. Additionally, fine-tuning the pretrained backbones (BERT and ResNet) on the movie domain could lead to better feature extraction specific to our task. The cold-start problem, in which a model attempts to predict ratings for a user with few to no ratings, would be an interesting direction to explore further. Finally, exploring similar but adjacent tasks, such as music recommendation (e.g., Spotify) or social recommendation (e.g., friend suggestions), could help extend our model into a more general-purpose recommendation system. Nonetheless, we believe this project lays a solid foundation for future exploration of scalable, multimodal recommendation systems.

Student Name	Contributed Aspects	Details
Tim Pang	Data cleaning, pre-processing, model implementation, system integration	Cleaned and joined datasets, engineered pre-processing data pipeline, implemented and trained multimodal model to get MSE/RMSE loss curves and prediction error charts, provided overall system integration.
Harrison Zhao	Data research and cleaning, model comparison experiment	Researched and compiled the datasets used in our model and performed initial data cleaning. Compared model to existing recommendation algorithms such as SVD and CF.
Michael Bock	Model research, ablation study	Investigated DLRM-based architectures and contributed to the design of our multimodal approach. Conducted and analyzed the ablation study to evaluate the impact of each modality on model performance.

Table 2. Contributions of team members.

References

- [1] carolzhangdc. Imdb 5000 movie dataset. <https://www.kaggle.com/datasets/carolzhangdc/imdb-5000-movie-dataset>, 2017. Accessed: 2025-04-28. **2**
- [2] Qiwei Chen, Weiping Liu, Yanru Shen, Hong Cheng, and Wei Wang. Addressing cold-start problem in click-through rate prediction via supervised diffusion modeling. *arXiv preprint arXiv:2306.09168*, 2023. **3**
- [3] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishu Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. Wide deep learning for recommender systems, 2016. **1**
- [4] F. Maxwell Harper and Joseph A. Konstan. The movielens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems (TiiS)*, 5(4):1–19, 2015. **2**
- [5] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. Neural collaborative filtering, 2017. **1**
- [6] jrobischo. Wikipedia movie plots dataset. <https://www.kaggle.com/datasets/jrobischo/wikipedia-movie-plots>, 2018. Accessed: 2025-04-28. **2**
- [7] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azcolini, Dmytro Dzhulgakov, Andrey Malleevich, Ilia Cherniavskii, Yinghai Lu, Raghuraman Krishnamoorthi, Ansha Yu, Volodymyr Kondratenko, Stephanie Pereira, Xianjie Chen, Wenlin Chen, Vijay Rao, Bill Jia, Liang Xiong, and Misha Smelyanskiy. Deep learning recommendation model for personalization and recommendation systems, 2019. **1, 2, 3**
- [8] neha1703. Movie genre from its poster. <https://www.kaggle.com/datasets/neha1703/movie-genre-from-its-poster>, 2018. Accessed: 2025-04-28. **2**
- [9] Jason Zhu, Yanling Cui, Yuming Liu, Hao Sun, Xue Li, Markus Pelger, Tianqi Yang, Liangjie Zhang, Ruofei Zhang, and Huasha Zhao. Textgcn: Improving text encoder via graph neural network in sponsored search. In *Proceedings of the Web Conference 2021*, WWW ’21, page 2848–2857. ACM, Apr. 2021. **1**